

Classification Transformer for Block Size & Stabilizing Steps

Date: 2026-05-05

Motivation

The current PhaseTransformer uses MSE regression on normalized continuous values to predict (block_size, refinement_steps). Trace data analysis reveals three structural properties that MSE regression cannot capture:

1. **Block size is trimodal** — 19.4 % are 1 (phase boundary sentinels), 28.2 % are 16 (AdaBlock default), and the remaining ~52 % spread across 2–128 with a geometric tail. MSE predicts the conditional mean, which falls in no-man's-land (~8–9) and is never correct for either regime.
2. **Stabilizing steps are determined by block size** — block_size = 1 always has max_stab_step = 0. Larger blocks have higher stab steps monotonically but with increasing variance. Predicting stab steps without conditioning on block size discards the most informative signal.
3. **Strong sequential structure** — traces alternate between content blocks and boundary sentinels (1). Top transitions: 1 → 16 (8 114), 16 → 16 (6 645), 16 → 1 (1 860). 93.9 % of 1s are preceded by a value > 1.

Design

Output Head Architecture

Replace the single regression head with two task-specific heads sharing the same Transformer encoder:

Block size head. `Linear(d_model → 128)` produces logits over classes 1–128. Argmax at inference. Cross-entropy loss captures the multimodal distribution naturally — the model learns that 1, 16, and every other value are distinct choices, not points on a continuum.

Stab steps head. Ordinal regression with K-1 binary outputs where $K = 83$ (max observed max_stab_step + 1). Each output models $P(\text{max_stab_step} > k)$ for $k = 0 \dots 81$. The first threshold $P(> 0)$ acts as an automatic zero-gate (38.2 % of training stab steps are 0). At inference, prediction = count of thresholds where $\text{sigmoid}(\text{logit}) > 0.5$. Loss: BCEWithLogitsLoss summed over all K-1 outputs; each training example contributes to every threshold.

Input Features

12 features per tuple, all with < 95 % zero values:

Feature	Description
block_size	Tokens in block (0–128)
nfe	Number of function evaluations
mean_stab_step	Mean stabilization step across tokens in block
max_stab_step	Max stabilization step
mean_ref_step	Mean refinement steps
max_ref_step	Max refinement steps
mean_gap	Mean gap (refinement minus stabilization)
max_gap	Max gap
mean_top1_confidence	Mean top-1 token probability
min_top1_confidence	Min top-1 token probability (low = hard)
digit_fraction	Fraction of digit tokens in block
delimiter_fraction	Fraction of delimiter tokens

All 12 features from past tuples in the context window are fed as input (past block_size and max_stab_step are historical values, not future leakage). Features are normalized to zero-mean unit-variance per field.

Encoder

No architectural change to the encoder itself:

- `window_size = 8` (default)
- `d_model = 64` , `n_heads = 4` , `n_layers = 2` , `dropout = 0.1`
- Sinusoidal positional encoding (Vaswani et al., 2017)
- Input projection: `Linear(input_tuple_size → d_model)`
- Last-position pooling for the encoding fed to both heads

Loss and Training

Combined loss:

`L = CrossEntropy(block_logits, block_target) + BCEWithLogits(stab_ordinals, stab_target)`

- Block target: integer `class_id ∈ [0, 127]` (mapping `1 → 0, ..., 128 → 127`)
- Stab target: binary vector of length 82 where `target[k] = 1` if `value > k` , else 0. For `value=0`, all elements are 0. For `value=5`, the first 5 elements are 1 and the rest 0.
- Equal weight ($\lambda = 1$) — both losses are in similar numerical range

Unchanged from current [train.py](#): AdamW (`lr=1e-3`, `weight_decay=1e-4`), `batch_size=32`, early stopping (`patience=10`), gradient clipping (`max_norm=1.0`), train/val split from end of sequence, checkpoint saving with best val loss.

Data source: `traces/rich/stab_tuples_conf_train_rich.jsonl`
(138 901 tuples across 5 000 sequences) and `test` equivalent.

Inference

- Model outputs `block_logits` (128) and `stab_logits` (82)
- Round `block_size` via `argmax` (no rounding ambiguity)
- Round `max_stab_step` via threshold counting (no rounding ambiguity)
- Predictor class updated to accept multi-head output and separate normalization statistics for input vs output fields

Why Not Alternative Approaches

- **Two-stage (boundary then content).** Adds complexity and error propagation. The 128-way classification head can learn that class 1 is special without explicit stage gating.
- **Larger encoder.** The current encoder (`d_model=64`, 2 layers) has sufficient capacity for 8-step sequence modeling on 12-d inputs. If

underfitting appears after the head change, scale `d_model` or `n_layers`.

- **Sequence-to-sequence / predict N steps.** Adds training complexity without clear benefit when downstream scheduler only needs the next tuple.
- **Bucketed block size.** Gives away the ability to predict exact values if needed later. 128-class classification on 138k tuples has adequate data per class.

Data Pipeline Changes

1. Load rich JSONL files via existing `extended_tuple_sequences_from_phase_tuples_jsonl` with `input_feature_fields` pointing to all 12 fields and `output_fields=("block_size", "max_stab_step")`.
2. Update `input_tuple_size = 12` in `ModelConfig`.
3. Update `output_tuple_size` remains 2 conceptually (`block_size`, `max_stab_step`) but the output head dimensions are (128, 82).

Model File Changes

- **`phase_predict/model.py`** — Replace single output head with two heads. Forward returns `(block_logits, stab_logits)`. Output head dimensions parameterized by `ModelConfig`.
- **`phase_predict/schema.py`** — Add `num_block_classes` and `num_stab_thresholds` to `ModelConfig`.
- **`phase_predict/train.py`** — Switch from `MSELoss` to combined cross-entropy + BCE loss. Update batch loop to handle two-output forward.
- **`phase_predict/predict.py`** — Update for multi-head output. Prediction rounding: `argmax` for `block_size`, threshold counting for `stab_steps`.
- **`phase_predict/dataset.py`** — Update target tensor construction for classification (int class label) and ordinal thresholds (binary vector).